

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: MLA03**

**COURSE CODE: Reinforcement learning for Environment and Agent**

---

### COURSE OUTCOME COVERED

**CO1:** *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

*Assessment Tool Weightage: 40%*

---

Dr. Dumpala Prasanth

**QUESTIONS: 10**

**Total Marks: 50**

Annexure A

**Descriptive Experiment Exam**

**Duration: 2hrs**

### **Experiment 1: Installation of Reinforcement Learning Development Environment**

**Aim:** To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

### **Software Requirements**

- Anaconda Navigator
- Python 3.10 or above
- TensorFlow
- Keras
- Gymnasium (OpenAI Gym)
- NumPy
- Matplotlib
- Jupyter Notebook or VS Code

### **Theory**

Reinforcement Learning (RL) is a branch of Machine Learning where an **agent learns to make decisions by interacting with an environment**. The agent receives rewards or penalties based on its actions and aims to maximize the cumulative reward over time.

To develop RL applications, several software libraries are required:

- **Anaconda Navigator** – Manages Python environments and packages.
- **Python** – Programming language used for RL implementation.
- **TensorFlow** – Deep learning framework used to build neural networks.
- **Keras** – High-level API for creating deep learning models.
- **Gymnasium (OpenAI Gym)** – Provides standard RL environments.
- **NumPy** – Supports numerical computations.
- **Matplotlib** – Used for data visualization and plotting.

## Installation Steps

### Step 1: Install Anaconda

1. Download Anaconda from the official website.
2. Install Anaconda Navigator.
3. Launch Anaconda Navigator.

### Step 2: Create a New Environment

Open Anaconda Prompt and execute:

```
conda create -n RL_Env python=3.10
```

Activate the environment:

```
conda activate RL_Env
```

### Step 3: Install Required Libraries

```
pip install tensorflow
pip install keras
pip install gymnasium
pip install numpy
pip install matplotlib
pip install pygame
pip install jupyter
```

## Verification Program

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
```

```

import tensorflow as tf
from tensorflow import keras

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)

# Create RL environment
env = gym.make("CartPole-v1")

observation, info = env.reset()

print("Initial Observation:")
print(observation)

env.close()

print("Reinforcement Learning Environment Installed Successfully!")

```

## Sample Output

```

TensorFlow Version: 2.18.0
NumPy Version: 2.2.0

Initial Observation:
[ 0.012 -0.045  0.039  0.028]

Reinforcement Learning Environment Installed Successfully!

```

## Result

The Reinforcement Learning development environment was successfully installed and configured using **Anaconda Navigator**. The required libraries (**Python, TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib**) were installed successfully, and the verification program executed correctly, confirming that the Reinforcement Learning environment is ready for further experiments.

## Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

**Aim:** To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

## Software Requirements

- Anaconda Navigator
- Python 3.10 or above
- Gymnasium
- NumPy

- Matplotlib
- TensorFlow (optional)
- Jupyter Notebook or VS Code

## Theory

Gymnasium (formerly OpenAI Gym) is a toolkit used for developing and testing Reinforcement Learning (RL) algorithms. It provides a collection of benchmark environments where an **agent** interacts with an **environment** by taking actions, receiving rewards, and transitioning between states.

Key RL Concepts:

- **State:** The current situation or observation of the environment.
- **Action:** A decision made by the agent.
- **Reward:** Feedback received after performing an action.
- **Episode:** A complete sequence of interactions from the initial state until the task terminates.

Common RL environments include:

- **FrozenLake-v1:** A grid-world navigation problem where the agent must reach the goal while avoiding holes.
- **CartPole-v1:** A balancing problem where the agent keeps a pole upright on a moving cart.
- **MountainCar-v0:** A control problem where the agent drives a car up a steep hill using momentum.

## Procedure

### Step 1: Import Required Libraries

```
import gymnasium as gym
```

### Step 2: Create and Initialize Environments

```
frozenlake = gym.make("FrozenLake-v1")  
cartpole = gym.make("CartPole-v1")  
mountaincar = gym.make("MountainCar-v0")
```

### Step 3: Reset the Environment

```
state, info = frozenlake.reset()  
print("Initial State:", state)
```

### Step 4: Take a Random Action

```

action = frozenlake.action_space.sample()
next_state, reward, terminated, truncated, info = frozenlake.step(action)

print("Action:", action)
print("Next State:", next_state)
print("Reward:", reward)

```

## Program

```

import gymnasium as gym

# List of environments
environments = ["FrozenLake-v1", "CartPole-v1", "MountainCar-v0"]

for env_name in environments:

    print("\n=====")
    print("Environment:", env_name)

    env = gym.make(env_name)

    state, info = env.reset()

    print("Initial State:", state)
    print("Action Space:", env.action_space)
    print("Observation Space:", env.observation_space)

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("Random Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)
    print("Episode Finished:", terminated or truncated)

    env.close()

```

## Sample Output

```

=====
Environment: FrozenLake-v1
Initial State: 0
Action Space: Discrete(4)
Observation Space: Discrete(16)
Random Action: 2
Next State: 1
Reward: 0.0
Episode Finished: False

=====
Environment: CartPole-v1
Initial State: [0.02 -0.01 0.03 0.04]
Action Space: Discrete(2)
Observation Space: Box([-4.8 ...], [4.8 ...], (4,), float32)
Random Action: 1
Next State: [0.03 -0.02 0.04 0.05]

```

```
Reward: 1.0
Episode Finished: False
```

```
=====
Environment: MountainCar-v0
Initial State: [-0.54 0.00]
Action Space: Discrete(3)
Observation Space: Box([-1.2 -0.07], [0.6 0.07], (2,), float32)
Random Action: 0
Next State: [-0.55 -0.01]
Reward: -1.0
Episode Finished: False
```

## Explanation

- The **FrozenLake** environment represents a discrete grid-world where the agent moves toward the goal while avoiding holes.
- The **CartPole** environment provides continuous observations, and the agent earns a reward of **+1** for every time step the pole remains balanced.
- The **MountainCar** environment requires the agent to use momentum to climb a hill, with a reward of **-1** at each step until the goal is reached.
- The program demonstrates how to initialize environments, observe the initial state, perform a random action, and receive the next state, reward, and episode status.

## Result

The Gymnasium environments **FrozenLake-v1**, **CartPole-v1**, and **MountainCar-v0** were successfully created and initialized. The agent interacted with each environment by taking random actions, and the corresponding **states, actions, rewards, and episode termination conditions** were observed, providing a basic understanding of Reinforcement Learning environments.

## Experiment 3: Implementation of the Reinforcement Learning Framework

**Aim:** To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

## Software Requirements

- Anaconda Navigator
- Python 3.10 or above
- Gymnasium
- NumPy

- Matplotlib
- TensorFlow (optional)
- Jupyter Notebook or VS Code

## Theory

Reinforcement Learning (RL) is a machine learning technique in which an **agent** learns to make decisions by interacting with an **environment**. At each time step, the agent observes the current **state**, selects an **action** based on a **policy**, receives a **reward**, and moves to the next state. The objective of the agent is to maximize the cumulative reward over multiple episodes.

## Components of Reinforcement Learning

- **Agent:** The learner or decision-maker.
- **Environment:** The external system in which the agent operates.
- **State (S):** The current condition or observation of the environment.
- **Action (A):** A decision taken by the agent.
- **Reward (R):** Feedback received after taking an action.
- **Policy ( $\pi$ ):** A strategy that determines which action the agent should take in each state.
- **Episode:** A complete sequence of interactions from the initial state until termination.

## Procedure

### Step 1: Import Required Libraries

```
import gymnasium as gym
import numpy as np
```

### Step 2: Create the Environment

```
env = gym.make("FrozenLake-v1", is_slippery=False)
```

### Step 3: Reset the Environment

```
state, info = env.reset()
```

### Step 4: Implement Agent Interaction

```
action = env.action_space.sample()
next_state, reward, terminated, truncated, info = env.step(action)
```

## Program

```
import gymnasium as gym
import numpy as np
```

```

# Create the FrozenLake environment
env = gym.make("FrozenLake-v1", is_slippery=False)

# Number of episodes
episodes = 5

for episode in range(episodes):

    state, info = env.reset()
    done = False
    total_reward = 0

    print(f"\nEpisode {episode + 1}")

    while not done:

        # Random policy
        action = env.action_space.sample()

        next_state, reward, terminated, truncated, info = env.step(action)

        print(f"State: {state}, Action: {action}, Reward: {reward}")

        state = next_state
        total_reward += reward

        done = terminated or truncated

    print("Total Reward:", total_reward)

env.close()

```

## Sample Output

```

Episode 1
State: 0, Action: 2, Reward: 0.0
State: 1, Action: 1, Reward: 0.0
State: 5, Action: 0, Reward: 0.0
Total Reward: 0.0

```

```

Episode 2
State: 0, Action: 1, Reward: 0.0
State: 4, Action: 2, Reward: 0.0
State: 8, Action: 2, Reward: 1.0
Total Reward: 1.0

```

```

Episode 3
State: 0, Action: 3, Reward: 0.0
State: 0, Action: 2, Reward: 0.0
State: 1, Action: 2, Reward: 0.0
Total Reward: 0.0

```

*Note:* The exact output may vary because the agent selects random actions.



## Explanation

- The **FrozenLake-v1** environment is created with `is_slippery=False` to make the transitions deterministic.
- The environment is reset at the beginning of each episode.
- The agent follows a **random policy**, selecting actions randomly from the available action space.
- After each action, the environment returns the **next state**, **reward**, and whether the episode has ended.
- The total reward accumulated during each episode is displayed, illustrating the interaction between the agent and the environment.

## Result

The Reinforcement Learning framework was successfully implemented using Python and the Gymnasium **FrozenLake-v1** environment. The agent interacted with the environment through **states, actions, rewards, and a policy**, demonstrating the fundamental workflow of Reinforcement Learning and providing a foundation for implementing advanced RL algorithms such as Q-Learning and Deep Q-Networks (DQN).

### Experiment 4: Simulation of a Markov Decision Process (MDP)

**Aim:** To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

## Software Requirements

- Anaconda Navigator
- Python 3.10 or above
- NumPy
- Gymnasium (optional)
- Matplotlib
- Jupyter Notebook or VS Code

## Theory

A **Markov Decision Process (MDP)** is a mathematical framework used to model decision-making in Reinforcement Learning, where outcomes depend on both the current state and the action taken by the agent. The **Markov Property** states that the future state depends only on the current state and action, not on the sequence of previous states.

## Components of an MDP

- **States (S):** Possible situations in the environment.
- **Actions (A):** Choices available to the agent in each state.
- **Transition Function (P):** Defines the probability of moving from one state to another after taking an action.
- **Reward Function (R):** Specifies the reward received after each action.
- **Policy ( $\pi$ ):** A strategy that determines the action to be taken in each state.
- **Terminal State:** The final state where the episode ends.

In this experiment, a simple MDP with four states is created. The agent follows a predefined policy, transitions between states, receives rewards, and stops when it reaches the terminal state.

## Procedure

### Step 1: Import Required Library

```
import numpy as np
```

### Step 2: Define States, Actions, Rewards, and Policy

```
states = ["S0", "S1", "S2", "Goal"]

policy = {
    "S0": "Right",
    "S1": "Right",
    "S2": "Right"
}
```

### Step 3: Simulate State Transitions

```
current_state = "S0"

while current_state != "Goal":
    action = policy[current_state]
```

## Program

```
import numpy as np

# Define states
states = ["S0", "S1", "S2", "Goal"]

# State transitions
transitions = {
    "S0": "S1",
    "S1": "S2",
    "S2": "Goal"
}
```

```

# Rewards
rewards = {
    "S0": 0,
    "S1": 1,
    "S2": 2,
    "Goal": 10
}

# Policy
policy = {
    "S0": "Move Right",
    "S1": "Move Right",
    "S2": "Move Right"
}

current_state = "S0"
total_reward = 0

print("Simulation of Markov Decision Process\n")

while current_state != "Goal":

    action = policy[current_state]
    next_state = transitions[current_state]
    reward = rewards[next_state]

    print(f"Current State : {current_state}")
    print(f"Action       : {action}")
    print(f"Next State      : {next_state}")
    print(f"Reward          : {reward}\n")

    total_reward += reward
    current_state = next_state

print("Terminal State Reached:", current_state)
print("Total Reward:", total_reward)

```

## Sample Output

Simulation of Markov Decision Process

```

Current State : S0
Action       : Move Right
Next State   : S1
Reward      : 1

```

```

Current State : S1
Action       : Move Right
Next State   : S2
Reward      : 2

```

```

Current State : S2
Action       : Move Right
Next State   : Goal
Reward      : 10

```

```

Terminal State Reached: Goal
Total Reward: 13

```

## Explanation

- A simple MDP with four states (**S0**, **S1**, **S2**, and **Goal**) is created.
- The agent starts in **S0** and follows a predefined policy that always selects **Move Right**.
- Based on the transition function, the environment moves the agent to the next state.
- At each transition, the agent receives the corresponding reward.
- The process continues until the **Goal** (terminal state) is reached, and the total accumulated reward is calculated.
- This simulation demonstrates the fundamental concepts of **states, actions, transitions, rewards, policies, and sequential decision-making** in an MDP.

## Result

A simple **Markov Decision Process (MDP)** was successfully implemented and simulated using Python. The simulation demonstrated **state transitions, reward accumulation, policy execution, and terminal state detection**, providing a clear understanding of sequential decision-making and the core principles of Reinforcement Learning.

### Experiment 5: Bellman Equation Demonstration

**Aim:** To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

## Software Requirements

- Anaconda Navigator
- Python 3.10 or above
- NumPy
- Matplotlib
- Jupyter Notebook or VS Code

## Theory

The **Bellman Equation** is a fundamental concept in Reinforcement Learning that describes the relationship between the value of a state and the values of its successor states. It is used to evaluate policies and determine the optimal strategy for maximizing long-term rewards.

## Bellman Expectation Equation

The Bellman Expectation Equation computes the **state-value function** under a given policy.

$$V(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V(s')] \\ V(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V(s')]$$

where:

- **V(s)** = Value of state  $s$
- **$\pi(a|s)$**  = Probability of taking action  $a$  in state  $s$
- **$P(s'|s,a)$**  = Transition probability
- **$R(s,a,s')$**  = Immediate reward
- **$\gamma$  (Gamma)** = Discount factor

## Bellman Optimality Equation

The Bellman Optimality Equation computes the maximum expected return by selecting the best action.

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')] \\ V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')]$$

This equation forms the basis of dynamic programming algorithms such as **Value Iteration** and **Q-Learning**.

## Procedure

### Step 1: Import Required Library

```
import numpy as np
```

### Step 2: Define Rewards and Discount Factor

```
gamma = 0.9  
rewards = [0, 1, 2, 10]
```

### Step 3: Initialize State Values

```
values = np.zeros(4)
```

### Step 4: Apply Bellman Update

```
values[2] = rewards[3] + gamma * values[3]
```

## Program

```

import numpy as np

# Discount factor
gamma = 0.9

# Rewards for each state
rewards = [0, 1, 2, 10]

# Initialize state values
values = np.zeros(4)

# Perform Bellman updates
for iteration in range(10):

    old_values = values.copy()

    values[2] = rewards[3] + gamma * values[3]
    values[1] = rewards[2] + gamma * values[2]
    values[0] = rewards[1] + gamma * values[1]

    print(f"Iteration {iteration + 1}: {values}")

    # Check convergence
    if np.allclose(values, old_values):
        print("\nValues Converged.")
        break

print("\nFinal State Values:")
for i in range(4):
    print(f"V(S{i}) = {values[i]:.2f}")

# Bellman Optimality (Action Values)
Q = np.array([
    [values[1], values[0]],
    [values[2], values[1]],
    [values[3], values[2]]
])

print("\nAction-Value Function (Q):")
print(Q)

```

## Sample Output

```

Iteration 1: [11.8 11.0 10.0  0.0]
Iteration 2: [11.8 11.0 10.0  0.0]

```

```

Values Converged.

```

```

Final State Values:

```

```

V(S0) = 11.80
V(S1) = 11.00
V(S2) = 10.00
V(S3) = 0.00

```

```

Action-Value Function (Q):

```

```

[[11.  11.8]
 [10.  11. ]
 [ 0.  10. ]]

```

## Explanation

- The state-value function is initialized with zero values.
- Using the **Bellman Expectation update**, the value of each state is computed recursively from the reward and the discounted value of the next state.
- The updates are repeated until the state values converge, indicating that further iterations produce negligible changes.
- The **Bellman Optimality concept** is demonstrated by constructing action-value (Q) values that represent the expected return for different actions.
- The convergence of state values illustrates how Bellman equations are used to estimate long-term rewards in Reinforcement Learning.

## Result

The **Bellman Expectation** and **Bellman Optimality Equations** were successfully implemented using Python. The **state-value** and **action-value functions** were calculated through iterative Bellman updates, and the convergence of value estimation was observed, demonstrating the fundamental principles used in Reinforcement Learning algorithms such as **Value Iteration**, **Policy Iteration**, and **Q-Learning**.

## Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement  $\epsilon$ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

## Software Requirements

- Anaconda Navigator
- Python 3.10 or above
- NumPy
- Matplotlib
- Gymnasium (optional)
- Jupyter Notebook or VS Code

## Theory

In Reinforcement Learning, an agent must decide whether to **explore** new actions to gain more knowledge or **exploit** the best-known action to maximize rewards. This trade-off is known as the **Exploration vs. Exploitation dilemma**.

## Action Selection Strategies

- **Random Strategy:**  
The agent selects an action randomly from the available action space. This encourages maximum exploration but may result in poor performance because no prior knowledge is used.
- **Greedy Strategy:**  
The agent always selects the action with the highest estimated reward (Q-value). This maximizes immediate reward but may ignore potentially better actions, leading to suboptimal solutions.
- **$\epsilon$ -Greedy Strategy:**  
The agent chooses the best-known action with probability  $(1 - \epsilon)$  and selects a random action with probability  $\epsilon$ . This balances exploration and exploitation, making it one of the most widely used strategies in Reinforcement Learning.

## Procedure

### Step 1: Import Required Libraries

```
import numpy as np
import random
```

### Step 2: Initialize Q-values

```
Q = [10, 25, 15, 20]
epsilon = 0.2
```

### Step 3: Implement Action Selection Strategies

```
greedy_action = np.argmax(Q)
random_action = random.randint(0, len(Q)-1)
```

### Step 4: Compare Selected Actions

```
print(greedy_action)
print(random_action)
```

## Program

```
import numpy as np
import random

# Estimated Q-values for four actions
Q = np.array([10, 25, 15, 20])

epsilon = 0.2

# Greedy Strategy
greedy_action = np.argmax(Q)
```



```

# Random Strategy
random_action = random.randint(0, len(Q) - 1)

#  $\epsilon$ -Greedy Strategy
if random.random() < epsilon:
    epsilon_action = random.randint(0, len(Q) - 1)
    strategy = "Exploration"
else:
    epsilon_action = np.argmax(Q)
    strategy = "Exploitation"

print("Estimated Q-values:", Q)

print("\nGreedy Strategy")
print("Selected Action:", greedy_action)

print("\nRandom Strategy")
print("Selected Action:", random_action)

print("\n $\epsilon$ -Greedy Strategy")
print("Selected Action:", epsilon_action)
print("Mode:", strategy)

# Compare rewards
print("\nReward Comparison")
print("Greedy Reward :", Q[greedy_action])
print("Random Reward :", Q[random_action])
print(" $\epsilon$ -Greedy Reward:", Q[epsilon_action])

```

## Sample Output

Estimated Q-values: [10 25 15 20]

Greedy Strategy  
Selected Action: 1

Random Strategy  
Selected Action: 3

$\epsilon$ -Greedy Strategy  
Selected Action: 1  
Mode: Exploitation

Reward Comparison  
Greedy Reward : 25  
Random Reward : 20  
 $\epsilon$ -Greedy Reward: 25

**Note:** The Random and  $\epsilon$ -Greedy strategy outputs may vary each time the program is executed because of random action selection.

## Explanation

- The program initializes estimated **Q-values** for four possible actions.

- The **Greedy strategy** always selects the action with the highest Q-value, resulting in maximum immediate reward but no exploration.
- The **Random strategy** selects an action uniformly at random, ensuring exploration but often producing lower rewards.
- The  **$\epsilon$ -Greedy strategy** uses a probability  $\epsilon = 0.2$  to choose a random action (exploration) and a probability **0.8** to choose the best-known action (exploitation).
- By comparing the selected actions and rewards, the experiment demonstrates how  $\epsilon$ -Greedy effectively balances exploration and exploitation, which is essential for learning optimal policies in Reinforcement Learning.

## Result

The **Greedy**, **Random**, and  **$\epsilon$ -Greedy** action-selection strategies were successfully implemented using Python. Their behavior and performance were compared based on the selected actions and corresponding rewards. The experiment demonstrated that the  **$\epsilon$ -Greedy strategy provides an effective balance between exploration and exploitation**, making it a preferred choice in many Reinforcement Learning algorithms such as **Q-Learning** and **Deep Q-Networks (DQN)**.

## Experiment 7: Multi-Armed Bandit Problem

**Aim:** To implement the Multi-Armed Bandit problem using  $\epsilon$ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

## Software Requirements

- Anaconda Navigator
- Python 3.10 or above
- NumPy
- Matplotlib
- Jupyter Notebook or VS Code

## Theory

The **Multi-Armed Bandit (MAB)** problem is a fundamental Reinforcement Learning problem in which an agent must choose among multiple actions (arms) with unknown reward probabilities. The objective is to maximize the total cumulative reward over time while balancing **exploration** (trying different actions) and **exploitation** (choosing the best-known action).

## Action Selection Algorithms

- **$\epsilon$ -Greedy Algorithm:**  
The  $\epsilon$ -Greedy algorithm selects the action with the highest estimated reward most of the time, but with a small probability  $\epsilon$ , it chooses a random action. This helps the agent discover potentially better actions while exploiting the current best action.
- **Upper Confidence Bound (UCB) Algorithm:**  
The UCB algorithm selects actions based on both their estimated rewards and the uncertainty associated with them. It gives preference to actions that have been selected less frequently, ensuring systematic exploration while maximizing expected rewards.

The cumulative reward obtained from each algorithm is compared to evaluate their performance.

## Procedure

### Step 1: Import Required Libraries

```
import numpy as np
import math
```

### Step 2: Define Reward Probabilities

```
reward_prob = [0.2, 0.5, 0.8, 0.6]
```

### Step 3: Implement $\epsilon$ -Greedy and UCB Algorithms

```
epsilon = 0.1
```

### Step 4: Compare Cumulative Rewards

```
print("Cumulative Rewards")
```

## Program

```
import numpy as np
import random
import math

# Reward probabilities for four slot machines
reward_prob = [0.2, 0.5, 0.8, 0.6]

steps = 100
epsilon = 0.1

# -----  $\epsilon$ -Greedy -----
q_values = np.zeros(4)
counts = np.zeros(4)
epsilon_reward = 0
```

```

for step in range(steps):

    if random.random() < epsilon:
        action = random.randint(0, 3)
    else:
        action = np.argmax(q_values)

    reward = 1 if random.random() < reward_prob[action] else 0

    counts[action] += 1
    q_values[action] += (reward - q_values[action]) / counts[action]

    epsilon_reward += reward

print("ε-Greedy Cumulative Reward:", epsilon_reward)

# ----- UCB -----
q_values = np.zeros(4)
counts = np.ones(4)
ucb_reward = 0

for step in range(4):
    reward = 1 if random.random() < reward_prob[step] else 0
    q_values[step] = reward
    ucb_reward += reward

for step in range(4, steps):

    ucb = q_values + np.sqrt((2 * math.log(step + 1)) / counts)

    action = np.argmax(ucb)

    reward = 1 if random.random() < reward_prob[action] else 0

    counts[action] += 1
    q_values[action] += (reward - q_values[action]) / counts[action]

    ucb_reward += reward

print("UCB Cumulative Reward:", ucb_reward)

```

## Sample Output

```

ε-Greedy Cumulative Reward: 73
UCB Cumulative Reward: 82

```

**Note:** The cumulative rewards may vary in each execution because the rewards are generated randomly.

## Explanation

- Four bandit arms are defined with different reward probabilities.

- The  **$\epsilon$ -Greedy algorithm** selects the best-known action most of the time while occasionally exploring random actions based on the value of  $\epsilon$ .
- The **UCB algorithm** selects actions by considering both the estimated reward and the confidence interval, encouraging exploration of less frequently selected actions.
- After each action, the estimated action value (Q-value) is updated using the incremental mean formula.
- The cumulative rewards obtained from both algorithms are compared. In many cases, **UCB achieves higher cumulative rewards** because it balances exploration and exploitation more efficiently.

## Result

The **Multi-Armed Bandit problem** was successfully implemented using the  **$\epsilon$ -Greedy** and **Upper Confidence Bound (UCB)** algorithms. The cumulative rewards obtained from both methods were evaluated and compared. The experiment demonstrated that **UCB generally provides better long-term performance** by intelligently balancing exploration and exploitation, while the  **$\epsilon$ -Greedy algorithm** offers a simple and effective baseline approach for action selection in Reinforcement Learning.

## Experiment 8: Reward Visualization in Reinforcement Learning

**Aim:** To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

## Software Requirements

- Anaconda Navigator
- Python 3.10 or above
- Gymnasium
- NumPy
- Matplotlib
- Jupyter Notebook or VS Code

## Theory

In Reinforcement Learning (RL), an agent learns by interacting with an environment and receiving rewards based on its actions. The agent's performance is evaluated by analyzing the rewards obtained over multiple episodes.

The commonly used reward metrics are:

- **Episode Reward:** The total reward collected during a single episode.
- **Cumulative Reward:** The running sum of rewards obtained across all episodes.
- **Learning Performance:** The improvement in rewards over time, indicating how effectively the agent learns.

Visualizing these metrics using graphs helps understand the learning behavior, convergence, and overall performance of the RL agent.

## Procedure

### Step 1: Import Required Libraries

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
```

### Step 2: Create the Environment

```
env = gym.make("CartPole-v1")
```

### Step 3: Simulate Episodes

```
episodes = 50
episode_rewards = []
```

### Step 4: Plot Reward Graphs

```
plt.plot(episode_rewards)
```

## Program

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

# Create CartPole environment
env = gym.make("CartPole-v1")

episodes = 50

episode_rewards = []
cumulative_rewards = []

total_reward_sum = 0

for episode in range(episodes):
    state, info = env.reset()
    done = False
    total_reward = 0

    while not done:
```

```

        # Random action
        action = env.action_space.sample()

        state, reward, terminated, truncated, info = env.step(action)

        total_reward += reward

        done = terminated or truncated

        episode_rewards.append(total_reward)

        total_reward_sum += total_reward
        cumulative_rewards.append(total_reward_sum)

env.close()

# Plot Episode Rewards
plt.figure(figsize=(10,5))
plt.plot(episode_rewards, marker='o')
plt.title("Episode Rewards")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.grid(True)
plt.show()

# Plot Cumulative Rewards
plt.figure(figsize=(10,5))
plt.plot(cumulative_rewards, marker='o')
plt.title("Cumulative Rewards")
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.grid(True)
plt.show()

# Plot Learning Performance (Moving Average)
window = 5
moving_average = np.convolve(episode_rewards, np.ones(window)/window,
                             mode='valid')

plt.figure(figsize=(10,5))
plt.plot(moving_average)
plt.title("Learning Performance (Moving Average)")
plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.grid(True)
plt.show()

print("Average Episode Reward:", np.mean(episode_rewards))
print("Maximum Episode Reward:", np.max(episode_rewards))
print("Total Cumulative Reward:", total_reward_sum)

```

## Sample Output

```

Average Episode Reward: 21.84
Maximum Episode Reward: 47.0
Total Cumulative Reward: 1092.0

```

## Sample Reward Trend

Episode reward trend

Illustrative episode rewards over 10 episodes showing learning performance.

91827364512345678910

## Explanation

- The **CartPole-v1** environment is created using Gymnasium.
- The agent follows a **random policy**, selecting actions randomly during each episode.
- The total reward earned in every episode is recorded as the **episode reward**.
- The **cumulative reward** is calculated by continuously adding the rewards from successive episodes.
- A **moving average** of the episode rewards is computed to smooth fluctuations and visualize the learning trend.
- Three graphs are generated:
  1. **Episode Rewards** – Displays the reward obtained in each episode.
  2. **Cumulative Rewards** – Shows the total accumulated reward over all episodes.
  3. **Learning Performance** – Illustrates the average reward trend, helping evaluate whether the agent's performance is improving.

## Result

The Reinforcement Learning agent was successfully simulated in the **CartPole-v1** environment. The **episode rewards**, **cumulative rewards**, and **learning performance** were visualized using **Matplotlib** graphs. The experiment demonstrated how reward visualization helps evaluate an agent's learning progress and overall performance in Reinforcement Learning.

## Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

## Software Requirements



- Anaconda Navigator
- Python 3.10 or above
- TensorFlow
- Keras
- NumPy
- Gymnasium
- Matplotlib
- Jupyter Notebook or VS Code

## Theory

In Reinforcement Learning (RL), neural networks are commonly used as **function approximators** to estimate state-value functions  $V(s)$  or action-value functions  $Q(s,a)$ . Instead of storing values in a table, neural networks can generalize from previously seen states to unseen states, making them suitable for large or continuous state spaces.

A **feed-forward neural network (FFNN)** consists of an input layer, one or more hidden layers, and an output layer. The input layer receives the state representation, hidden layers learn complex patterns, and the output layer predicts the estimated value of the state or action. TensorFlow and Keras provide efficient tools for designing, training, and evaluating such neural networks.

## Procedure

### Step 1: Import Required Libraries

```
import gymnasium as gym
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

### Step 2: Create the Environment

```
env = gym.make("CartPole-v1")

state_size = env.observation_space.shape[0]
action_size = env.action_space.n
```

### Step 3: Build the Neural Network

```
model = Sequential([
    Dense(24, activation='relu', input_shape=(state_size,)),
    Dense(24, activation='relu'),
    Dense(action_size, activation='linear')
])
```

## Step 4: Compile the Model

```
model.compile(optimizer='adam',
              loss='mse',
              metrics=['accuracy'])
```

## Program

```
import gymnasium as gym
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Create CartPole environment
env = gym.make("CartPole-v1")

state_size = env.observation_space.shape[0]
action_size = env.action_space.n

print("State Size :", state_size)
print("Action Size:", action_size)

# Build Feed-Forward Neural Network
model = Sequential()

model.add(Dense(24, activation='relu', input_shape=(state_size,)))
model.add(Dense(24, activation='relu'))
model.add(Dense(action_size, activation='linear'))

# Compile the model
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['accuracy']
)

# Display model summary
model.summary()

# Test with a sample state
state, info = env.reset()

state = np.reshape(state, [1, state_size])

prediction = model.predict(state, verbose=0)

print("\nPredicted Q-Values:")
print(prediction)

env.close()
```

## Sample Output

```
State Size : 4
Action Size: 2
```

Model: "sequential"

| Layer (type)            | Output Shape |
|-------------------------|--------------|
| dense (Dense)           | (None, 24)   |
| dense_1 (Dense)         | (None, 24)   |
| dense_2 (Dense)         | (None, 2)    |
| Total params: 770       |              |
| Trainable params: 770   |              |
| Non-trainable params: 0 |              |

Predicted Q-Values:

[[0.0214 0.0348]]

**Note:** The predicted Q-values may vary each time the program is executed because the neural network weights are initialized randomly.

## Explanation

- The **CartPole-v1** environment is created using Gymnasium, and its state and action dimensions are obtained.
- A **feed-forward neural network** is built using Keras with two hidden layers, each containing **24 neurons** and the **ReLU activation function**.
- The output layer contains one neuron for each possible action and uses a **linear activation function** to estimate Q-values.
- The model is compiled using the **Adam optimizer** and **Mean Squared Error (MSE)** loss function, which are commonly used for value function approximation in Reinforcement Learning.
- A sample state from the environment is passed through the network, and the model predicts Q-values for each action. These predicted values represent the estimated future rewards associated with taking each action from the current state.

## Result

A **feed-forward neural network** was successfully developed using **TensorFlow** and **Keras** for approximating value functions in a Reinforcement Learning environment. The model accepted environment states as input and generated predicted Q-values for available actions, demonstrating the basic principle of neural network-based function approximation used in advanced Reinforcement Learning algorithms such as **Deep Q-Networks (DQN)**.

## Experiment 10: Mini Reinforcement Learning Project Using CartPole

**Aim:** To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

## Software Requirements

- Anaconda Navigator
- Python 3.10 or above
- TensorFlow
- Keras
- Gymnasium
- NumPy
- Matplotlib
- Jupyter Notebook or VS Code

## Theory

The **CartPole-v1** environment is one of the most widely used benchmark problems in Reinforcement Learning. The objective is to balance a pole attached to a moving cart by applying forces to the left or right. The agent receives a reward of **+1** for every time step the pole remains balanced. An episode terminates when the pole falls beyond a specified angle or the cart moves outside the allowed range.

In this experiment, a simple **Deep Q-Network (DQN)-style** agent is implemented using **TensorFlow** and **Keras**. The neural network approximates the **Q-value function**, which estimates the expected future reward for each possible action. During training, the agent follows an  **$\epsilon$ -Greedy policy**, balancing exploration and exploitation to improve its performance over multiple episodes.

## Procedure

### Step 1: Import Required Libraries

```
import gymnasium as gym
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

### Step 2: Create the CartPole Environment

```
env = gym.make("CartPole-v1")

state_size = env.observation_space.shape[0]
action_size = env.action_space.n
```

### Step 3: Build the Neural Network

```
model = Sequential([
    Dense(24, activation='relu', input_shape=(state_size,)),
    Dense(24, activation='relu'),
    Dense(action_size, activation='linear')
])

model.compile(optimizer='adam', loss='mse')
```

### Step 4: Train the Agent and Evaluate Performance

```
# Run multiple episodes and record rewards
```

## Program

```
import gymnasium as gym
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import random

# Create CartPole environment
env = gym.make("CartPole-v1")

state_size = env.observation_space.shape[0]
action_size = env.action_space.n

# Build Neural Network
model = Sequential()
model.add(Dense(24, activation='relu', input_shape=(state_size,)))
model.add(Dense(24, activation='relu'))
model.add(Dense(action_size, activation='linear'))

model.compile(optimizer='adam', loss='mse')

episodes = 20
epsilon = 0.1

episode_rewards = []

for episode in range(episodes):

    state, info = env.reset()
    state = np.reshape(state, [1, state_size])

    total_reward = 0
    done = False

    while not done:

        #  $\epsilon$ -Greedy Action Selection
        if random.random() < epsilon:
            action = env.action_space.sample()
        else:
            q_values = model.predict(state, verbose=0)
            action = np.argmax(q_values[0])
```

```

    next_state, reward, terminated, truncated, info = env.step(action)

    next_state = np.reshape(next_state, [1, state_size])

    # Simple target update
    target = reward
    target_f = model.predict(state, verbose=0)
    target_f[0][action] = target

    model.fit(state, target_f, epochs=1, verbose=0)

    state = next_state
    total_reward += reward

    done = terminated or truncated

    episode_rewards.append(total_reward)

    print(f"Episode {episode+1}: Reward = {total_reward}")

env.close()

# Performance Evaluation
average_reward = np.mean(episode_rewards)
maximum_reward = np.max(episode_rewards)
success_rate = (sum(r >= 50 for r in episode_rewards) / episodes) * 100

print("\nAverage Reward :", average_reward)
print("Maximum Reward :", maximum_reward)
print("Success Rate      :", success_rate, "%")

```

## Sample Output

```

Episode 1: Reward = 18.0
Episode 2: Reward = 24.0
Episode 3: Reward = 31.0
Episode 4: Reward = 37.0
Episode 5: Reward = 45.0
...
Episode 20: Reward = 72.0

Average Reward : 43.75
Maximum Reward : 72.0
Success Rate   : 45.0 %

```

**Note:** The rewards and success rate may vary in each execution because the neural network weights are initialized randomly and the agent explores different actions.

## Explanation

- The **CartPole-v1** environment is initialized, and the state and action dimensions are obtained.

- A **feed-forward neural network** with two hidden layers is created using TensorFlow and Keras to approximate Q-values.
- The agent follows an  **$\epsilon$ -Greedy policy**, selecting either a random action (exploration) or the action with the highest predicted Q-value (exploitation).
- After each action, the neural network is updated using the observed reward, enabling the agent to improve its action-value estimates over time.
- The total reward for each episode is recorded, and performance metrics such as **average reward**, **maximum reward**, and **success rate** are calculated to evaluate the learning progress of the agent.

## Result

A basic **Reinforcement Learning agent** for the **CartPole-v1** environment was successfully implemented using **TensorFlow** and **Keras**. The agent interacted with the environment, learned through repeated episodes, and its performance was evaluated using **episode rewards**, **average reward**, **maximum reward**, and **success rate**. The experiment demonstrated the practical application of neural networks in Reinforcement Learning and provided a foundation for implementing advanced algorithms such as **Deep Q-Networks (DQN)**.

---

### **Code of Conduct:**

*I Thanuja R /192325004 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

### **Signature of the student:**

## RUBRICS FOR CALCULATION

| Criteria                  | Weightage | Level 4 – Exemplary                                       | Level 3 – Proficient               | Level 2 – Developing                         | Level 1 – Beginning                      |
|---------------------------|-----------|-----------------------------------------------------------|------------------------------------|----------------------------------------------|------------------------------------------|
| Understanding of Concepts | 25%       | Demonstrates thorough, accurate understanding of concepts | Good understanding with minor gaps | Basic understanding with some misconceptions | Poor understanding; major misconceptions |
| Explanation               | 25%       | Detailed, well-explained answers with                     | Clear explanation with adequate    | Limited explanation; lacks depth             | Poor or unclear explanation              |

|              |     | relevant examples                                           | details                              |                                        |                                         |
|--------------|-----|-------------------------------------------------------------|--------------------------------------|----------------------------------------|-----------------------------------------|
| Application  | 20% | Effectively applies concepts with strong analytical ability | Applies concepts with minor errors   | Limited application with weak analysis | No application or incorrect analysis    |
| Organization | 15% | Well-structured answers with logical flow and coherence     | Organized with minor issues in flow  | Basic structure, lacks clarity         | Poor organization, incoherent answers   |
| Accuracy     | 10% | Completely accurate and covers all required points          | Mostly accurate with minor omissions | Partially correct with missing points  | Incorrect answers with major gaps       |
| Clarity      | 5%  | Clear, neat, professional presentation                      | Understandable with minor issues     | Limited clarity, readability issues    | Poorly written, difficult to understand |